



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

INGENIERÍA CIVIL
TELEMÁTICA

Teoría de Sistemas Operativos

ELO321 - Módulo 1

Lección 7

Docente: Berioska Contreras Vargas
MSc Security and Privacy – EIT/UNITN/UTU



■ Tabla de Contenidos

Tópicos

Evaluación de Algoritmos

Fórmula de Little

Problema Sección Crítica

Solución de Peterson

Variables Atómicas



Evaluación de Algoritmos

- **Objetivo:** elegir un planificador de CPU para un S.O.
 - Determinar un criterio implica evaluar los algoritmos.
- Modelamiento **determinista** o evaluación **analítica**.
 - Considera una carga de trabajo predeterminada.

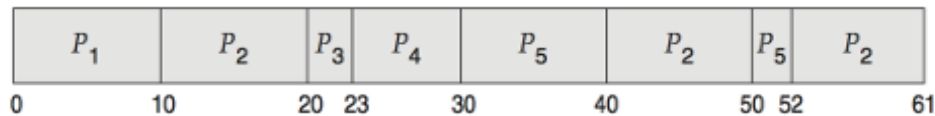
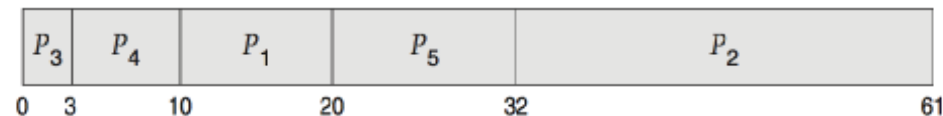
Ejemplo:

Proceso	Tiempo de Ráfaga
P1	10
P2	29
P3	3
P4	7
P5	12



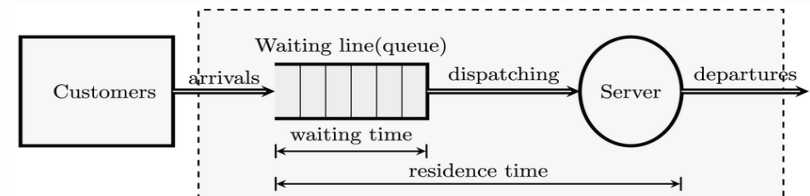
Ejemplo: Evaluación Determinista

- Por cada algoritmo se calcula el promedio de tiempo de espera mínimo. Requiere números exactos como entradas. Ejemplo:
- FCFS: 28ms, SJF: 13 ms, RR: 28ms.



Modelos Encolamiento

- Describe propabilísticamente el arribo de procesos y tiempos de ráfaga y procesador.
 - exponencial, promedio.
 - media de uso, espera, etc.
- Un sistema computacional consiste de una red de servicios, cada uno con una fila de espera.
 - Conoce la tasa de arribo.
 - Calcula uso, duración, espera, etc.





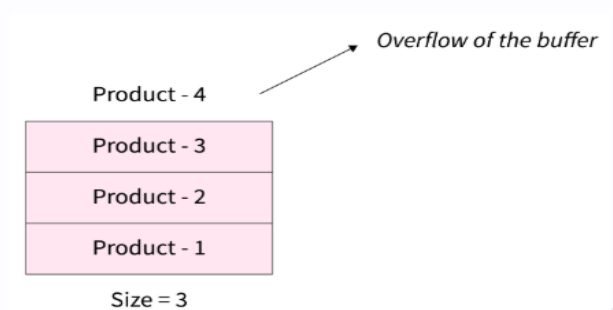
Fórmula de Little

- En estado estacionario, los procesos que salen de la cola deben ser iguales a los procesos que llegan, y así:
 - $\eta = \lambda * \omega$, donde
 - η longitud promedio de la cola.
 - ω tiempo promedio de espera en la cola.
 - λ tasa promedio de llegada a la cola.
- **Ejemplo:** si en promedio llegan **7** procesos por segundo y normalmente hay **14** procesos en la cola, entonces el tiempo promedio de espera por proceso sería de 2 segundos.



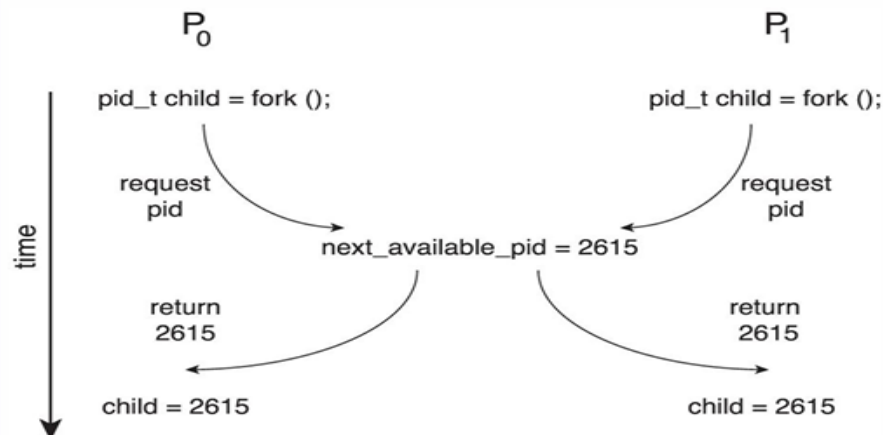
Problema Sección Crítica I

- Procesos se pueden ejecutar concurrentemente.
 - No excentos de interrupciones o ejecuciones parciales.
- Acceso concurrente a datos compartidos puede resultar en inconsistencia de datos.
- Una ejecución ordenada debe ser consistente.
- Recordar caso "buffer" sin límite.



Condición de Carrera

- Procesos **P₀** y **P₁** crean un proceso hijo a través de la función **fork()**.
- Condición de carrera: en variable `next_available_pid`, nada previene la asignación del mismo identificador a **P₀** y **P₁**.





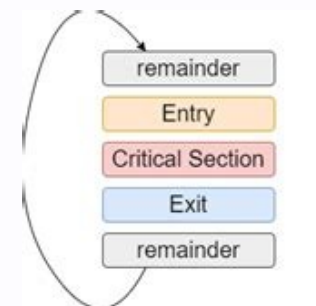
Problema Sección Crítica II

- Sea una serie de $P_n = \{P_0, P_1, \dots, P_{(n-1)}\}$
- Cada proceso tiene una sección de código que representa una sección crítica:
 - Procesos cambiando variables, actualizando tablas, escribiendo archivo, etc.
- El problema de la sección crítica es un desafío de diseño donde cada proceso debe solicitar acceso para ingresar a ella, salir o reingresar.

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

Requerimientos

- **Exclusión Mutua:** Si P1 ejecuta la sección crítica, los procesos distintos a P1 no pueden ejecutarla.
- **Progreso:** Una vez disponible la sección crítica, un proceso debe ser seleccionado para ingresar sin ser postpuesto indefinidamente.
- **Espera limitada:** Existe un límite de veces que otros procesos ingresan a la sección crítica, después de solicitar acceso y luego de ser otorgado.



Solución por Interrupciones

- **Sección ingreso:** interruptor de desactivación
- **Sección salida:** interrupción de activación.
- Desafíos:
 - tiempo de ejecución
 - procesos en espera infinita
 - múltiples procesadores
- Solución 1
 - Dos procesos **atómicos**.
 - `int turn # var compartida`
 - `turn = i # en sección crítica`

```
while (true){  
  
    while (turn == j);  
  
    /* critical section */  
  
    turn = j;  
  
    /* remainder section */  
  
}
```



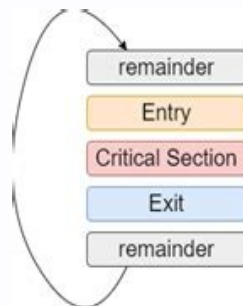
Solución de Peterson

- Dos procesos atómicos.
- Dos variables compartidas.
 - int turn; # a sección crítica
 - boolean flag[2] # listo para sección crítica
- flag[i]=true # proceso listo

```
while (true){  
  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
  
    /* critical section */  
  
    flag[i] = false;  
  
    /* remainder section */  
  
}
```

Arquitectura Moderna

- Peterson reúne los 3 requerimientos:
 - **Exclusión mutua** $\text{flag}[j]=\text{false}$ or $\text{turn}=i$.
 - **Progreso** es satisfecho
 - **Límite de espera** definido.
- Solución pensada para un un hilo único.
- En arquitecturas con **múltiples hilos** el reordenamiento podría causar **inconsistencias**.



```
static void *increment_thread(void *arg)
{
    int niters = (int)arg;
    for (int i = 0; i < niters; i++) {
        ?
        shared_counter++;
        ?
    }
    return (void *)0;
}
```



■ Ejemplo: Multihilos

- Dos hilos comparten datos.
- ¿Cuál es el valor de salida esperado?
- Considerar variables independientes y el reordenamiento de variables.

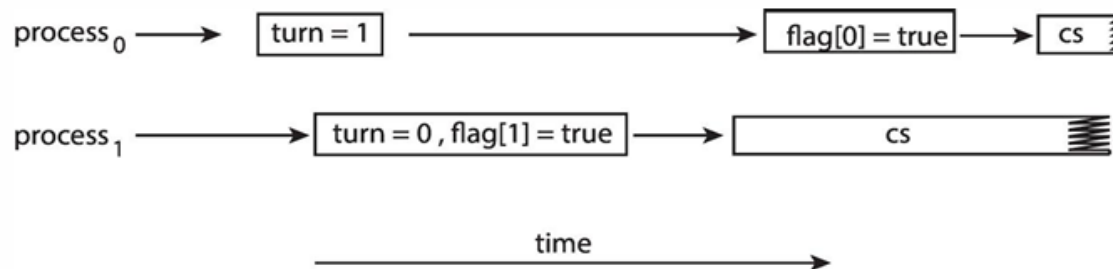
```
boolean flag = false;  
int x = 0;
```

```
Thread 1 performs  
while (!flag)  
;  
print x
```

```
Thread 2 performs  
x = 100;  
flag = true
```

Peterson y Multihilos

- Los efectos del **reordenamiento** de variables.
- Permite a los procesos estar en la sección crítica al mismo tiempo.
- Se necesita una **Barrera de Memoria**.



Barrera de Memoria

- **Modelo de memoria ordenado de manera fuerte** cuando la modificación de un proceso es visible inmediatamente por todos los procesos.
- **Modelo de memoria ordenado de manera débil** cuando la modificación de un proceso no es visible inmediatamente por todos los procesos.
- Una **Barrera de Memoria** es una instrucción que fuerza cualquier cambio y su propagación a los procesadores.

```
Thread 1 now performs
while (!flag)
    memory_barrier();
print x

Thread 2 now performs
x = 100;
memory_barrier();
flag = true
```




Implementaciones

- **Procesador unitario** desactiva los interruptores, permitiendo la ejecución de código sin prevención.
- Instrucciones físicas:
 - test and set
 - compare and swap
- **Variables atómicas:**
 - sin interrupciones.

Thread 1:	Thread 2:
<code>start_atomic() {</code>	<code>start_atomic() {</code>
<code> load X</code>	<code> load X</code>
<code> X = X + 1</code>	<code> X = X + 1</code>
<code> store X</code>	<code> store X</code>
<code>end_atomic()</code>	<code>end_atomic()</code>

Instrucción Test & Set

- **Propiedades:**

- Ejecución atómica.
- Retorna el valor original de un parámetro traspasado.
- Define un nuevo valor, true.

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}

do {
    while (test_and_set(&lock))
        ; /* do nothing */

    /* critical section */

    lock = false;
    /* remainder section */
} while (true);
```

Instrucción Compare & Swap

- **Propiedades:**

- Ejecución atómica.
- Retorna el valor original de un parámetro traspasado.
- Define un nuevo valor, siempre que el valor esperado es verdadero.

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}

while (true){
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}
```

Variables Atómicas

- Proveen actualizaciones **sin interrupciones** en tipo de datos básicos como enteros y booleanos.
 - Ejemplo: `increment(&sequence);`

```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v, temp, temp+1)));
}
```



♥ ¡Gracias por tu Atención!

```
$ echo "Initiating gentle system halt..."  
$ sudo shutdown -h now
```